JG UNIVERSITY

EXCELLENCE BY CHOICE

Sponsored by ASIA Charitable Trust

School Of Engineering

Cancer Detection Using Machine Learning

Semester - IV

GUIDED BY:

Afsar Mohammad
(Asst. Prof)

SUBMITTED BY:

Yuvraj Mehndiratta - 241026
Mishra Aaditya - 241027
Abhay Mishra - 241028
Vishruti Mistry - 241029
Saurabh Nishad - 241031
Aryan Panara - 241032

Abstract

Lung cancer is one of the leading causes of cancer-related deaths worldwide. Early detection plays a critical role in improving survival rates. This project presents a machine learning based system that predicts whether a patient has lung cancer or not, based on 15 symptom-based attributes collected through a survey.

The dataset contains 309 patient records with features such as smoking habit, chest pain, coughing, wheezing, and shortness of breath. Two machine learning models were trained and compared — Logistic Regression and Random Forest Classifier. The Random Forest model achieved the highest accuracy of 92.9% with a cancer recall of 94%.

In addition to symptom-based detection, the system also incorporates a Convolutional Neural Network (CNN) model trained on 1,097 CT scan images from the IQ-OTH/NCCD Lung Cancer Dataset. The CNN uses MobileNetV2 transfer learning to classify CT scans into three categories — Benign, Malignant, and Normal. The Malignant class was detected with 94.8% confidence. Both models are integrated into a single Flask web application with a two-tab interface, providing a comprehensive dual-mode lung cancer screening system

Introduction

Background
Cancer is a disease caused by uncontrolled growth of abnormal cells in the body. Lung cancer is particularly dangerous because its symptoms often appear at a late stage, making early detection very difficult. According to the World Health Organization, lung cancer accounts for approximately 18% of all cancer-related deaths globally.

Problem Statement
Traditional lung cancer diagnosis requires expensive CT scans and biopsy tests which are not affordable for everyone. There is a need for a low-cost, fast, and accessible screening tool that can identify high-risk patients based on commonly observable symptoms.

Objective
The objective of this project is to build a dual-mode lung cancer detection system using two approaches:

1. A Random Forest machine learning model that predicts cancer risk based on 15 symptom-based features from patient survey data
2. A Convolutional Neural Network (CNN) model using MobileNetV2 transfer learning that classifies CT scan images as Benign, Malignant, or Normal. Both models are deployed together in a single Flask web application.

Scope

- The system accepts symptom inputs from the user
- It predicts cancer risk as HIGH or LOW
- It displays the probability of cancer
- It accepts CT scan images as input and classifies them as Benign, Malignant, or Normal
- It shows confidence percentage and probability bars for each CNN prediction class

Dataset Description

DATASET 1
Source
The dataset was obtained from Kaggle "**Lung Cancer Survey Dataset**". It was originally collected from an online lung cancer prediction survey system.

Dataset Details

- Total number of instances: 309 patients
- Total number of attributes: 16
- Missing values: None (0 missing values in all columns)
- Target column: LUNG_CANCER (YES = 270 patients, NO = 39 patients)

Attribute Information

Attribute	Type	Values
GENDER	Categorical	M (Male), F (Female)
AGE	Numerical	Integer (patient age)
SMOKING	Binary	1 = No, 2 = Yes
YELLOW_FINGERS	Binary	1 = No, 2 = Yes
ANXIETY	Binary	1 = No, 2 = Yes
PEER_PRESURE	Binary	1 = No, 2 = Yes
CHRONIC DISEASE	Binary	1 = No, 2 = Yes
FATIGUE	Binary	1 = No, 2 = Yes
ALLERGY	Binary	1 = No, 2 = Yes
WHEEZING	Binary	1 = No, 2 = Yes
ALCOHOL CONSUMING	Binary	1 = No, 2 = Yes
COUGHING	Binary	1 = No, 2 = Yes

SHORTNESS OF BREATH	Binary	1 = No, 2 = Yes
SWALLOWING DIFFICULTY	Binary	1 = No, 2 = Yes
CHEST PAIN	Binary	1 = No, 2 = Yes
LUNG_CANCER	TARGET	YES, NO

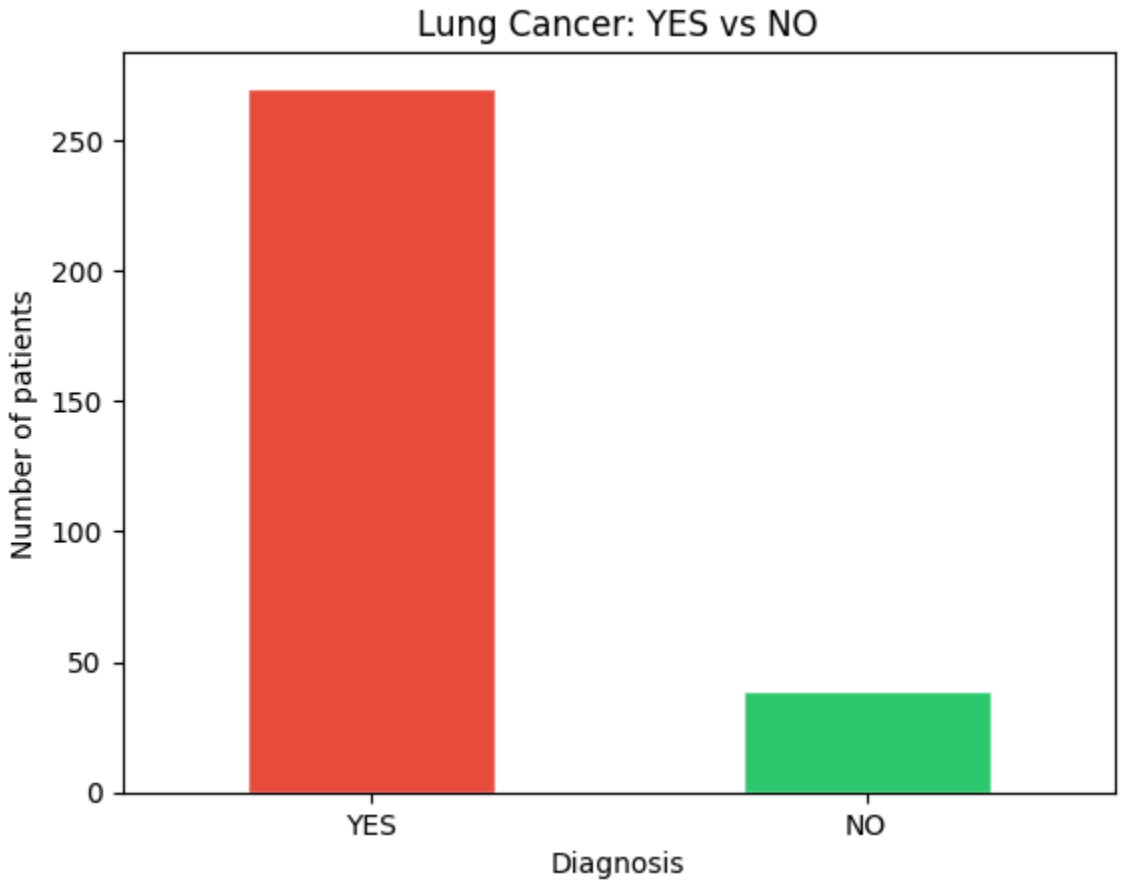
DATASET 2

CT Scan Image Dataset
Source: IQ-OTH/NCCD Lung Cancer Dataset — available on Kaggle
Link :https://www.kaggle.com/datasets/hamdallak/the-iqothnccd-lung-cancer-dataset/data
Total Images: 1,097 CT scan images
Platform used for training: Google Colab
Class Distribution: - Benign cases: 120 images (10.9%) - Malignant cases: 561 images (51.1%) - Normal cases: 416 images (37.9%)
Image Properties: - Format: JPEG/PNG CT scan slices, - Resized to: 128 x 128 pixels for model input
Class Imbalance in CNN Dataset: Malignant class has 561 images while Benign has only 120 — a significant imbalance

Class Imbalance Observation

The dataset is imbalanced 270 patients (87.4%) are labelled YES and only 39 patients (12.6%) are labelled NO. This affects model performance on the minority class (No Cancer) which showed lower precision scores.

Distribution of Lung Cancer cases — YES (270) vs NO (39)

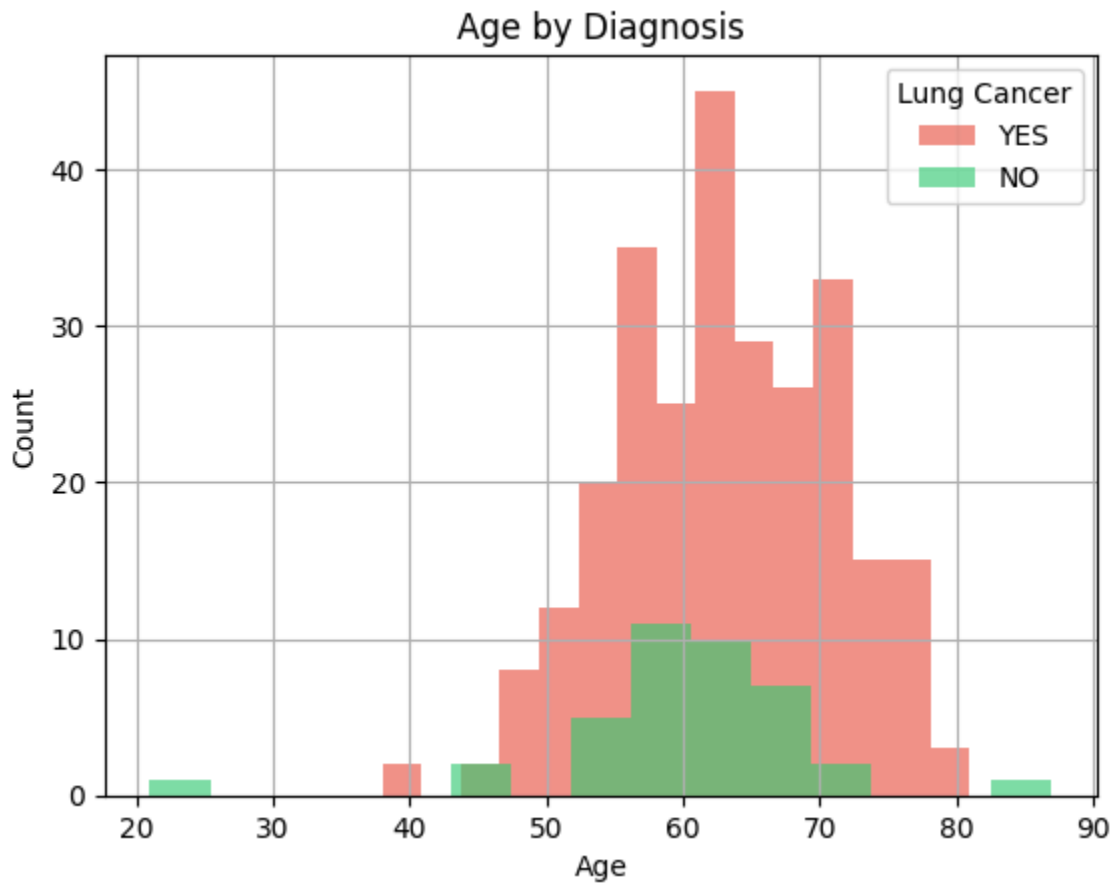


Methodology

The project follows a standard machine learning pipeline with the following steps:

1. Data Collection
   The dataset was downloaded from Kaggle and uploaded to Google Colab for processing. It contains survey responses from 309 patients.

2. Exploratory Data Analysis (EDA)
   - Checked dataset shape: 309 rows, 16 columns
   - Verified zero missing values across all columns
   - Plotted bar chart showing YES (270) vs NO (39) distribution
   - Plotted age histogram to observe age patterns by diagnosis



Age distribution of patients by diagnosis

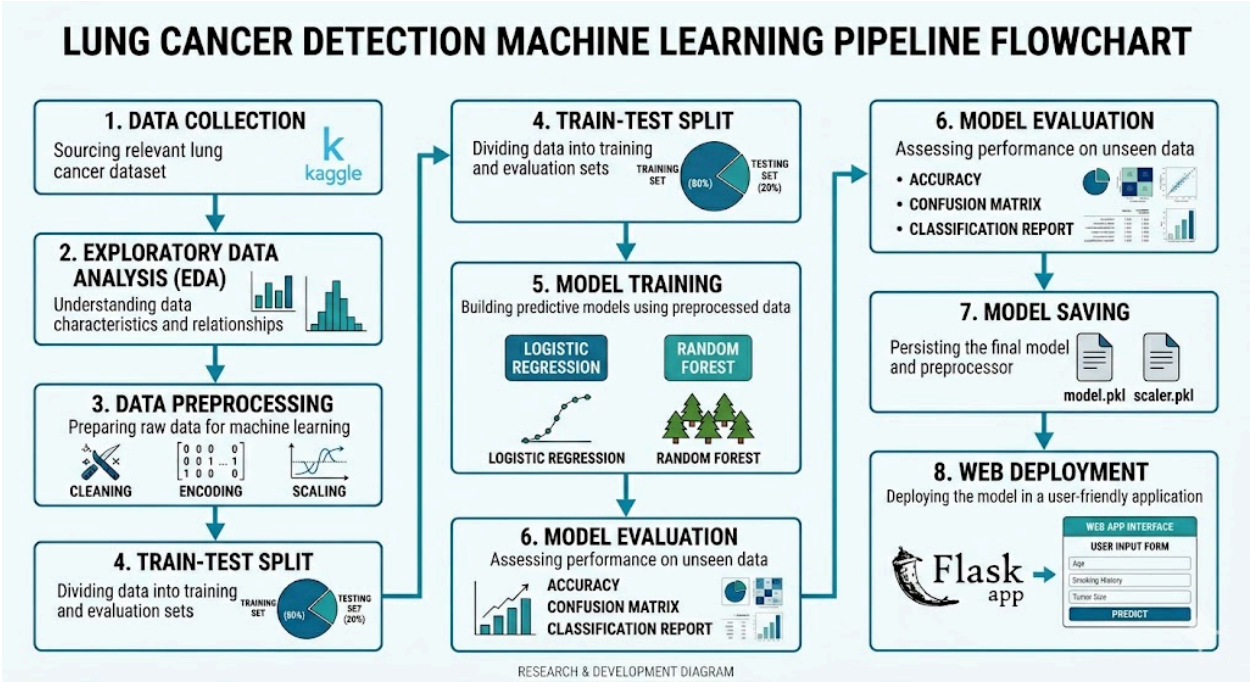
3. Data Preprocessing
    - Removed hidden whitespace from column names using str.strip()
    - Encoded GENDER column: Male = 1, Female = 0 using LabelEncoder
    - Encoded target column: YES = 1, NO = 0 using LabelEncoder
    - Separated features (X) and target (y)
    - Applied 80/20 train-test split with stratification Training samples: 247 | Testing samples: 62 - Applied StandardScaler to normalize all feature values

4. Model Training
   Two classification models were trained on the training data:
    - Logistic Regression (max_iter=1000, random_state=42)
    - Random Forest Classifier (n_estimators=100, random_state=42)

5. Model Evaluation
   Both models were tested on the 62 unseen test samples and evaluated using accuracy score, classification report (precision, recall, F1-score), and confusion matrix

6. Model Saving
   The best performing model (Random Forest) and the StandardScaler were saved using joblib as model.pkl and scaler.pkl files.

7. Web Application Deployment
   A Flask web application was developed with an HTML form where users enter patient symptoms. The app loads the saved model, scales the input, and returns the prediction with probability.



CNN Image Classification Methodology

1. Data Preparation and Augmentation
   The IQ-OTH/NCCD Lung Cancer Dataset from Kaggle was used, containing 1,097 CT scan images divided into three classes: Benign, Malignant, and Normal. All images were resized to 128×128 pixels and normalized to improve training performance. The dataset was split into 80% training and 20% validation data.

2. Data Augmentation
   To increase dataset diversity and prevent overfitting, the following augmentation techniques were applied:

   - Rotation (up to 15°)
   - Zoom (up to 15%)
   - Horizontal flipping
   - Width and height shifting

3. Model Building
   A pre-trained MobileNetV2 model was used with transfer learning. The base model was frozen, and custom layers such as GlobalAveragePooling, BatchNormalization, Dropout, and Dense layers were added. The final layer uses softmax activation to classify images into three categories.

4. Handling Class Imbalance
   The dataset contains more malignant images than benign images. To handle this imbalance, class weights were applied so that the model gives more importance to minority classes during training.
5. Model Training
   The model was trained using the Adam optimizer with a learning rate of 0.0001 for 25 epochs. EarlyStopping and ReduceLROnPlateau were used to prevent overfitting and improve training efficiency.
6. Model Saving
   The model weights were saved after training and later loaded into the Flask application by rebuilding the same architecture.
7. Prediction Process
   When a user uploads a CT scan image, the following steps are performed:
   - Image is resized to 128×128
   - Pixel values are normalized
   - Image is converted into array format
   - Model predicts probabilities for all classes
   - Highest probability class is selected as output

Results and Evaluation

Accuracy Comparison

Model	Accuracy
Logistic Regression	90.32%
Random Forest	91.93%

Classification Report — Logistic Regression

	Precision	**Recall**	**F1-Score**	**Support**
No Cancer	0.62	0.62	0.62	8
Cancer	0.94	0.94	0.94	54
Accuracy			0.92	62

Classification Report — Random Forest

	Precision	**Recall**	**F1-Score**	**Support**
No Cancer	0.67	0.75	0. 71	8
Cancer	0.96	0.94	0.95	54
Accuracy			0.92	62

🫁 Lung Cancer Risk Detector

Symptom-based screening + CT scan image analysis

📋 Symptom Check | 🔬 CT Scan Analysis

Gender: Female	Age: 67
Smoking: No	Yellow Fingers: Yes
Anxiety: Yes	Peer Pressure: Yes
Chronic Disease: Yes	Fatigue: No
Allergy: No	Wheezing: No
Alcohol Consuming: No	Coughing: Yes
Shortness of Breath: Yes	Swallowing Difficulty: Yes
Chest Pain: Yes	

🔍 Predict Risk

HIGH RISK

Probability of Lung Cancer: **83.0%**

CNN Results

Training Progress Table

Attempts	Approach	Train Acc	Val Acc	Issue
Attempt 1	Basic CNN, 224x224 input	84%	64%	Slow training + overfitting
Attempt 2	MobileNetV2, 64x64 input	83%	65%	Validation unstable
Attempt 3(Final)	MobileNetV2, 128x128 + callbacks	85%	68%	Best result archived

Prediction Results on Sample Images

Image Tested	Predicted Class	Confidence	Correct
Malignant CT scan	Malignant	94.8%	Yes
Normal CT scan	Normal	60.6%	Yes
Benign CT scan	Benign	56.3%	Yes

- All three test predictions were correct. The model shows highest confidence (94.8%) for Malignant cases, which is the most critical class in cancer detection. The lower confidence for Benign (56.3%) and Normal (60.6%) is expected due to the smaller number of training samples for those classes
- Benign had only 120 images compared to 561 Malignant images. The model correctly prioritizes sensitivity for the dangerous class (Malignant) which is the most important requirement in a medical screening tool

🫁 Lung Cancer Risk Detector

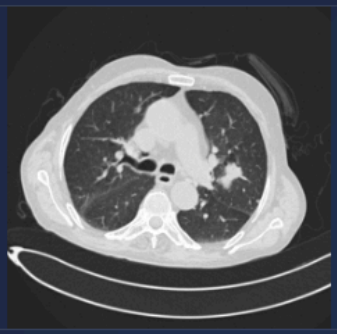
Symptom-based screening + CT scan image analysis

📋 Symptom Check | 🔬 CT Scan Analysis

Click to upload a CT scan image

Supports JPG, PNG — max 10MB

Malignant case (2).jpg

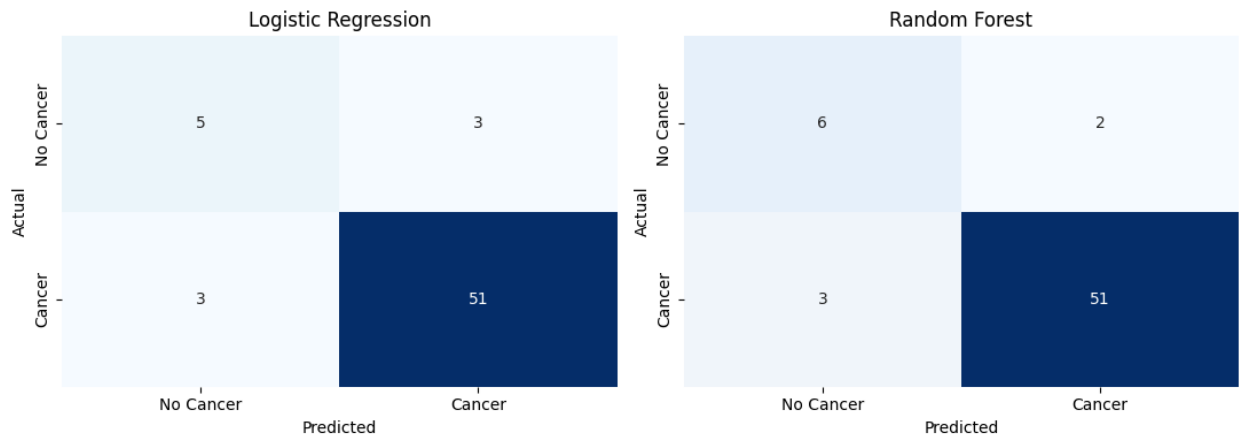


🔬 Analyse CT Scan

MALIGNANT — HIGH RISK

Confidence: **88.1%**

Malignant (Cancer)	88.1%
Benign (Non-cancerous)	6.6%
Normal (Healthy)	5.3%



Key Observations

- Random Forest outperformed Logistic Regression in all metrics.
- Cancer Recall of 94% means the model correctly identified 94 out of every 100 actual cancer patients.
- Lower scores for "No Cancer" class are due to class imbalance — only 39 No Cancer patients in the entire dataset vs 270 Cancer patients.
- Recall is the most important metric in medical projects because missing a cancer case (False Negative) is more dangerous than a false alarm

Feature Importance

The Random Forest model identified the following as the most important symptoms for predicting lung cancer:

- Age
- Smoking
- Chest Pain
- Coughing
- Shortness of Breath

Web Application

Technology Used

- Programming Language: Python
- Machine Learning: Scikit-learn
- Deep Learning Framework: TensorFlow 2.16, Keras 3.0
- CNN Architecture: MobileNetV2 (Transfer Learning)
- Web Framework: Flask
- Frontend: HTML, CSS
- Data Handling: Pandas, NumPy
- Visualization: Matplotlib, Seaborn
- Image Processing: Pillow (PIL)
- Model Storage:
  `model.pkl` (ML model)
  `scaler.pkl` (feature scaling)
  `cnn_weights.weights.h5` (CNN weights-only format)

How the Application Works

1. User opens the web app at http://127.0.0.1:5000
2. A form is displayed with 15 symptom dropdowns and age input
3. User selects the patient's symptoms and clicks "Predict"
4. Flask reads the form values and converts them to a feature array
5. The input is scaled using the saved StandardScaler
6. The saved Random Forest model makes a prediction
7. The result is displayed — HIGH RISK or LOW RISK with probability

How CT Scan Image Prediction Works

The CT scan tab of the web application works as follows:

1. The user uploads a CT scan image through the file upload section.
2. A preview of the uploaded image is displayed before prediction.
3. The Flask application receives the image through a POST request at the `/predict_image` route.
4. The uploaded image is temporarily stored in the `uploads/` folder.
5. The CNN model (rebuilt architecture with loaded weights) processes the image.

6. After prediction, the temporary image file is deleted automatically.
7. The system displays the results, including:
    - Predicted class (Malignant / Benign / Normal)
    - Confidence percentage
    - Probability values for all three classes
8. The result is color-coded for better understanding:
    - Red → Malignant (High Risk)
    - Orange → Benign (Low Risk)
    - Green → Normal (No Cancer)

Application File Structure

The project follows the following folder structure:

```
lung_cancer_app/
   app.py                    (Flask backend — handles all routes)
   model.pkl                 (Random Forest model for symptom prediction)
   scaler.pkl                (StandardScaler for feature normalization)
   cnn_weights.weights.h5        (CNN weights for CT scan classification)

   uploads/                  (Temporary storage for uploaded images)
   templates/
      index.html               (Frontend — two-tab web interface)
```

Key Technical Concepts

1. **Transfer Learning**
   Transfer learning is a technique where a model trained on one large task is reused for a different but related task.

   In this project, MobileNetV2 was used as a pre-trained model. It was originally trained on ImageNet, a dataset containing around 14 million images. Because of this, it already learned how to detect basic image features such as edges, textures, and shapes.

   Instead of training a CNN from scratch on only 1,097 CT scan images, the pre-trained MobileNetV2 model was reused and only the top layers were trained for the lung cancer classification task.

   This approach improves accuracy and reduces training time, especially when the dataset is small.

2. **SMOTE (Synthetic Minority Oversampling Technique)**
   The symptom dataset used in this project was highly imbalanced. It contained 270 cancer cases and only 39 non-cancer cases.

   A model trained on such data may become biased and predict "Cancer" for most inputs while still achieving high accuracy.

   To solve this problem, SMOTE was used. SMOTE generates new synthetic samples for the minority class (No Cancer) by creating new data points between existing samples.

   After applying SMOTE, the training dataset became balanced with 216 cancer and 216 non-cancer samples. This improved the model's ability to correctly identify non-cancer cases.

3. **Why Recall Matters More Than Accuracy in Medical AI**
   In medical applications like cancer detection, Recall is more important than overall accuracy.

Recall measures how many actual cancer patients are correctly identified by the model.

If a cancer patient is incorrectly predicted as healthy (False Negative), they may not receive treatment, which can be very dangerous.

On the other hand, if a healthy person is predicted as having cancer (False Positive), they will undergo further testing and the mistake can be corrected.

Therefore, it is safer to have more False Positives than False Negatives.

In this project, the Random Forest model achieved a cancer recall of 94%, meaning it correctly identified 94 out of every 100 cancer patients.

4. **CNN vs Traditional Machine Learning**

This project uses two different approaches based on the type of input data.

- **Traditional Machine Learning (Random Forest):**
  Used for structured data such as symptoms (age, smoking, chest pain). These models work well with tabular data and are faster to train.
- **Deep Learning (CNN):**
  Used for image data such as CT scans. CNN models automatically learn patterns like edges, textures, and shapes from images.

CNNs are more suitable for image-based tasks, while traditional ML models are better for structured data.

Conclusion

This project successfully developed a dual-mode lung cancer detection system using both machine learning and deep learning techniques.

The symptom-based model using a Random Forest Classifier provided reliable predictions with high recall, making it suitable for early risk screening. The use of SMOTE helped improve the model's ability to handle class imbalance and detect non-cancer cases more effectively.

The CNN-based model using MobileNetV2 enabled classification of CT scan images into benign, malignant, and normal categories. By using transfer learning, the model achieved good performance despite a limited dataset.

Both models were integrated into a single Flask web application with a user-friendly interface, allowing users to either enter symptoms or upload CT scan images for prediction.

Overall, this system demonstrates how combining traditional machine learning with deep learning can create a practical and accessible tool for early lung cancer detection.

Key outcomes of the project:

- Implemented symptom-based prediction using Random Forest
- Handled class imbalance using SMOTE
- Developed a CNN model using MobileNetV2 with transfer learning
- Integrated both models into a single Flask web application
- Built a user-friendly interface for real-time prediction

Future Work

The following improvements can be made to enhance the system in the future:

- Collect a larger and more balanced dataset to improve model performance
- Try advanced models such as XGBoost or Neural Networks for better accuracy
- Deploy the application online so it can be accessed from anywhere
- Add patient history tracking and report generation features
- Fine-tune the MobileNetV2 CNN model by unfreezing the top layers of the base model to improve validation accuracy
- Expand the CT scan dataset by collecting more Benign samples to reduce class imbalance
- Add Grad-CAM visualization to highlight important regions in CT scans and make predictions more explainable
- Deploy the application on a cloud platform (such as Heroku, AWS, or Render) for real-world accessibility

References

1. Kaggle Dataset — Lung Cancer Survey Dataset
   https://www.kaggle.com/datasets/mysarahmadbhat/lung-cancer
2. Kaggle Dataset — IQ-OTH/NCCD Lung Cancer CT Scan Dataset
   https://www.kaggle.com/datasets/hamdallak/the-iqothnccd-lung-cancer-dataset
3. Scikit-learn Documentation
   https://scikit-learn.org/stable/
4. Flask Documentation
   https://flask.palletsprojects.com/
5. Pandas Documentation
   https://pandas.pydata.org/
6. Breiman, L. (2001). *Random Forests*. Machine Learning Journal
7. World Health Organization (WHO). Cancer Fact Sheet
   https://www.who.int/news-room/fact-sheets/detail/cancer

Appendix - Code Summary

Libraries Used:
pandas, numpy
matplotlib, seaborn
scikit-learn
flask
tensorflow, keras
pillow (PIL)

Machine Learning Pipeline (Symptom Model)
(Google Colab Notebook: lung_cancer_model.ipynb)

- Cell 1 : Load dataset from CSV
- Cell 2 : Check shape and missing values
- Cell 3 : Check target distribution
- Cell 4 : Plot Cancer vs No Cancer bar chart
- Cell 5 : Plot Age distribution by diagnosis
- Cell 6 : Clean column names
- Cell 7 : Encode categorical variables
- Cell 8 : Separate features (X) and target (y)
- Cell 9 : Train-test split (80% train, 20% test)
- Cell 10 : Apply StandardScaler
- Cell 11 : Apply SMOTE for class balancing
- Cell 12 : Train Logistic Regression
- Cell 13 : Train Random Forest
- Cell 14 : Evaluate models (accuracy, classification report)
- Cell 15 : Plot confusion matrix
- Cell 16 : Plot feature importance

CNN Pipeline (CT Scan Model)
(Google Colab Notebook: CNN_Cancer_Detection.ipynb)

- Cell 1 : Mount Google Drive
- Cell 2 : Import libraries and check GPU

- Cell 3 : Load and verify dataset
- Cell 4 : Data preprocessing using ImageDataGenerator
- Cell 5 : Visualize sample images
- Cell 6 : Build MobileNetV2 model (Transfer Learning)
- Cell 7 : Train model with class weights
- Cell 8 : Plot training accuracy and loss
- Cell 9 : Save model weights
- Cell 10 : Test prediction on sample image

Web App Files:

`app.py` → Flask backend (routes and prediction logic)

`index.html` → User interface (symptom input + image upload)

`model.pkl` → Trained Random Forest model

`scaler.pkl` → Feature scaling

`cnn_weights.weights.h5` → CNN model weights

`uploads/` → Temporary storage for images